

● Étape 1 : Définition du problème

- **Type de modèle** : Supervisé – Classification binaire.
- **Objectif** : prédire si un client est **solvable (1)** ou **non solvable (0)** à partir de ses données financières (revenus, historique de paiement, dettes).
- **Pourquoi ?** : utile pour les banques/assurances afin de réduire les risques de crédit.

👉 Ici, ta variable cible sera **solvable** (0/1).

● Étape 2 : Préparation des données

a) Chargement et exploration

```
import pandas as pd
```

```
data = pd.read_csv("credit_data.csv")
print(data.head())
print(data.info())
print(data['solvable'].value_counts())
```

b) Prétraitement

- **Encodage** : transformer les variables catégorielles (**historique_paiement**, **type_contrat**) en numériques (OneHotEncoder ou LabelEncoder).
- **Normalisation** : mettre les variables continues (**revenus**, **dettes**) sur la même échelle (StandardScaler).

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
# Exemple encodage
```

```
data['historique_paiement'] = LabelEncoder().fit_transform(data['historique_paiement'])
```

```
# Normalisation
```

```
scaler = StandardScaler()
```

```
data[['revenus', 'dettes']] = scaler.fit_transform(data[['revenus', 'dettes']])
```

● Étape 3 : Split Train/Test

```
from sklearn.model_selection import train_test_split
```

```
X = data.drop('solvable', axis=1)
y = data['solvable']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)
```

● Étape 4 : Modélisation

a) Régression logistique

```
from sklearn.linear_model import LogisticRegression
```

```
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
y_pred_log = log_model.predict(X_test)
```

b) Arbre de décision

```
from sklearn.tree import DecisionTreeClassifier
```

```
tree_model = DecisionTreeClassifier(max_depth=5, random_state=42)
tree_model.fit(X_train, y_train)
y_pred_tree = tree_model.predict(X_test)
```

● Étape 5 : Évaluation

- **Matrice de confusion**
- **Précision, rappel, F1-score**

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Logistic Regression
print("Régression Logistique")
print(classification_report(y_test, y_pred_log))
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_log), annot=True, fmt='d')
plt.title("Matrice de confusion - Régression Logistique")
plt.show()
```

```
# Decision Tree
print("Arbre de Décision")
```

```
print(classification_report(y_test, y_pred_tree))

sns.heatmap(confusion_matrix(y_test, y_pred_tree), annot=True, fmt='d')
plt.title("Matrice de confusion - Arbre de Décision")
plt.show()
```

👉 Tu compares les deux modèles et choisis celui qui équilibre le mieux **précision** et **rappel**.

● Étape 6 : Application Streamlit

Exemple de squelette

```
import streamlit as st
import joblib
import pandas as pd

# Charger modèle choisi
model = joblib.load("best_credit_model.pkl")

st.title("Prédiction du score de crédit")

revenus = st.number_input("Revenus mensuels", min_value=0)
dettes = st.number_input("Montant des dettes", min_value=0)
historique = st.selectbox("Historique de paiement", ["Bon", "Moyen", "Mauvais"])

if st.button("Prédire"):
    # Prétraitement identique au notebook
    data = pd.DataFrame([[revenus, dettes, historique]],
                        columns=["revenus", "dettes", "historique_paiement"])
    prediction = model.predict(data)[0]
    st.write("Résultat :", "Solvable ✅" if prediction==1 else "Non solvable ❌")
```

Très bien 🏆, commençons par poser une **structure claire du projet en MVC**. Cela va t'aider à organiser ton Notebook et ton application Streamlit de manière professionnelle et maintenable.

📁 Structure MVC du projet – Prédiction du score de crédit

● 1. Model (M)

👉 Partie qui gère les **données et les algorithmes ML**.

- Chargement et exploration du dataset.
- Prétraitement : encodage des variables catégorielles, normalisation des variables numériques.
- Split train/test.
- Entraînement des modèles (régression logistique, arbre de décision).
- Évaluation de confusion, précision, rappel, F1-score).
- Sauvegarde du modèle choisis modèles (matrice de `joblib`).

📁 Organisation :

/model

preprocessing.py # fonctions d'encodage, normalisation

train.py # entraînement des modèles

evaluate.py # métriques et matrice de confusion

save_model.py # sauvegarde du modèle

● 2. View (V)

👉 Partie qui gère l'**interface utilisateur** (Streamlit).

- Formulaire de saisie des données financières (revenus, dettes, historique de paiement).
- Bouton de prédiction → affiche solvable / non solvable.
- Visualisation de la matrice de confusion (heatmap).

📁 Organisation :

/view

app.py # application Streamlit

components.py # fonctions pour affichage (formulaire, graphiques)

● 3. Controller (C)

👉 Partie qui fait le lien entre **Model et View**.

- Récupère les données saisies par l'utilisateur.
- Applique le même prétraitement que dans le modèle.
- Charge le modèle sauvegardé.
- Retourne la prédiction et les métriques à la View.

📁 Organisation :

/controller

predict.py # fonction qui prend les inputs et renvoie la prédiction

pipeline.py # orchestre prétraitement + modèle

🟢 Workflow global

1. **Model** : Notebook → prétraitement, entraînement, sauvegarde du modèle.
 2. **Controller** : fonction `predict_credit_score()` qui prend les inputs du formulaire, applique le prétraitement, et renvoie la prédiction.
 3. **View** : Streamlit → formulaire + affichage matrice de confusion.
-

💡 Exemple d'arborescence finale :

projet_credit/

|

|— model/

| |— preprocessing.py

| |— train.py

| |— evaluate.py

| |— save_model.py

|

|— controller/

| |— predict.py

| |— pipeline.py

```
|
|— view/
|  |— app.py
|  |— components.py
|
|— notebook_credit.ipynb # Notebook Jupyter documenté
```